



Section 9 / 24

[Go to Questions](#) ↓

Debugging with GDB

Now that we have the general information about our program, we will start running it and debugging it. Debugging consists mainly of four steps:

Step	Description
Break	Setting breakpoints at various points of interest
Examine	Running the program and examining the state of the program at these points
Step	Moving through the program to examine how it acts with each instruction and with user input
Modify	Modify values in specific registers or addresses at specific breakpoints, to study how it would affect the execution

We will go through these points in this section to learn the basics of debugging a program with GDB.

Break

The first step of debugging is setting `breakpoints` to stop the execution at a specific location or `when a particular condition is met`. This helps us in examining the state of the program and the value of registers at that point. `Breakpoints` also allow us to stop the program's execution at that point so that we can step into each instruction and examine how it changes the program and values.

We can set a breakpoint at a specific address or for a particular function. To set a breakpoint, we can use the `break` or `b` command along with the address or function name we want to break at. For example, to follow all instructions run by our program, let's break at the `_start` function, as follows:

```
gef> b _start
```

```
Breakpoint 1 at 0x401000
```

shellsession

Now, in order to start our program, we can use the `run` or `r` command:

```
gef> b _start
```

```
Breakpoint 1 at 0x401000
```

```
gef> r
```

```
Starting program: ./helloWorld
```

```
Breakpoint 1, 0x0000000000401000 in _start ()
```

```
[ Legend: Modified register | Code | Heap | Stack | String ]
```

```
registers ——— $rax : 0x0
```

```
$rbx : 0x0
```

```
$rcx : 0x0
```

```

$rdx      : 0x0
$rsp      : 0x00007fffffff310 → 0x0000000000000001
$rbp      : 0x0
$rsi      : 0x0
$rdi      : 0x0
$rip      : 0x0000000000401000 → <_start+0> mov eax, 0x1
...SNIP...

```

```

stack ——— 0x00007fffffff310 | +0x0000: 0x0000000000000001 ←
$rsp0x00007fffffff318 | +0x0008: 0x00007fffffff5a0 → "./helloWorld"
...SNIP...

```

```

code:x86:64 ———
      0x400ffa      add     BYTE PTR [rax], al
      0x400ffc      add     BYTE PTR [rax], al
      0x400ffe      add     BYTE PTR [rax], al
→ 0x401000 <_start+0>    mov     eax, 0x1
      0x401005 <_start+5>    mov     edi, 0x1
      0x40100a <_start+10>   movabs  rsi, 0x402000
      0x401014 <_start+20>   mov     edx, 0x12
      0x401019 <_start+25>   syscall
      0x40101b <_start+27>   mov     eax, 0x3c

```

```

threads ———
[#0] Id 1, Name: "helloWorld", stopped 0x401000 in _start (), reason:
BREAKPOINT
trace ———
[#0] 0x401000 → _start()

```

If we want to set a breakpoint at a certain address, like `<_start+10>`, we can either `b <_start+10>` or `b *0x40100a`:

```

gef> b *0x40100a
Breakpoint 1 at 0x40100a

```

shellsession

The `*` tells `GDB` to break at the instruction stored in `0x40100a` .

Note: Once the program is running, if we set another breakpoint, like `b *0x401005` , in order to continue to that breakpoint, we should use the `continue` or `c` command. If we use `run` or `r` again, it will run the program from the start. This can be useful to skip loops, as we will see later in the module.

If we want to see what breakpoints we have at any point of the execution, we can use the `info breakpoint` command. We can also `disable` , `enable` , or `delete` any breakpoint. Furthermore, GDB also supports setting conditional breaks that stop the execution when a specific condition is met.

Examine

The next step of debugging is `examining` the values in registers and addresses. As we can see in the previous terminal output, `GEF` automatically gave us a lot of helpful information when we hit our breakpoint. This is one of the benefits of having the `GEF` plugin, as it automates many steps that we usually take at every breakpoint, like examining the registers, the stack, and the current assembly instructions.

To manually examine any of the addresses or registers or examine any other, we can use the `x` command in the format of `x/FMT ADDRESS` , as `help x` would tell us. The `ADDRESS` is the address or register we want to examine, while `FMT` is the examine format. The examine format `FMT` can have three parts:

Argument	Description	Example
Count	The number of times we want to repeat the examine	2 , 3 , 10
Format	The format we want the result to be represented in	x(hex) , s(string) , i(instruction)
Size	The size of memory we want to examine	b(byte) , h(halfword) , w(word) , g(giant, 8 bytes)

Instructions

For example, if we wanted to examine the next four instructions in line, we will have to examine the `$rip` register (which holds the address of the next instruction), and use `4` for the `count` , `i` for the `format` , and `g` for the `size` (for 8-bytes or 64-bits). So, the final examine command would be `x/4ig $rip` , as follows:

```
shellsession
gef> x/4ig $rip

=> 0x401000 <_start>:  mov    eax,0x1
    0x401005 <_start+5>: mov    edi,0x1
    0x40100a <_start+10>: movabs rsi,0x402000
    0x401014 <_start+20>: mov    edx,0x12
```

We see that we get the following four instructions as expected. This can help us as we go through a program in examining certain areas and what instructions they may contain.

Strings

We can also examine a variable stored at a specific memory address. We know that our `message` variable is stored at the `.data` section on address `0x402000` from our previous disassembly. We also see the upcoming command `movabs rsi, 0x402000`, so we may want to examine what is being moved from `0x402000`.

In this case, we will not put anything for the `Count`, as we only want one address (1 is the default), and will use `s` as the format to get it in a string format rather than in hex:

```
shellsession
gef> x/s 0x402000

0x402000:  "Hello HTB Academy!"
```

As we can see, we can see the string at this address represented as text rather than hex characters.

Note: if we don't specify the `Size` or `Format`, it will default to the last one we used.

Addresses

The most common format of examining is hex `x`. We often need to examine addresses and registers containing hex data, such as memory addresses, instructions, or binary data. Let us examine the same previous instruction, but in `hex` format, to see how it looks:

```
shellsession
gef> x/wx 0x401000

0x401000 <_start>:  0x000001b8
```

We see instead of `mov eax,0x1`, we get `0x000001b8`, which is the hex representation of the `mov eax,0x1` machine code in little-endian formatting.

- This is read as: `b8 01 00 00`.

Try repeating the commands we used for examining strings using `x` to examine them in hex. We should see the same text but in hex format. We can also use `GEF` features to examine certain addresses. For example, at any point we can use the `registers` command to print out the current value of all registers:

```
gef➤ registers
$rax : 0x0
$rbx : 0x0
$rcx : 0x0
$rdx : 0x0
$rsp : 0x00007fffffffe310 → 0x0000000000000001
$rbp : 0x0
$rsi : 0x0
$rdi : 0x0
$rip : 0x0000000000401000 → <_start+0> mov eax, 0x1
...SNIP...
```

Step

The third step of debugging is `stepping` through the program one instruction or line of code at a time. As we can see, we are currently at the very first instruction in our `helloWorld` program:

```
code:x86:64 ———
0x400ffe          add    BYTE PTR [rax], al
→ 0x401000 <_start+0>  mov    eax, 0x1
0x401005 <_start+5>    mov    edi, 0x1
```

Note: the instruction shown with the `->` symbol is where we are at, and it has not yet been processed.

To move through the program, there are three different commands we can use: `stepi` and `step`.

Step Instruction

The `stepi` or `si` command will step through the assembly instructions one by one, which is the smallest level of steps possible while debugging. Let us use the `si` command to see how we get to the next instruction:

```
code:x86:64 ———gef> si
0x0000000000401005 in _start ()
0x400fff          add    BYTE PTR [rax+0x1], bh
→ 0x401005 <_start+5>  mov    edi, 0x1
0x40100a <_start+10>   movabs rsi, 0x402000
0x401014 <_start+20>   mov    edx, 0x12
0x401019 <_start+25>   syscall
```

```
threads ———
[#0] Id 1, Name: "helloWorld", stopped 0x401005 in _start (), reason: SINGLE STEP
```

As we can see, we took exactly one step and stopped again at the `mov edi, 0x1` instruction.

Step Count

Similarly to examine, we can repeat the `si` command by adding a number after it. For example, if we wanted to move 3 steps to reach the `syscall` instruction, we can do so as follows:

```
gef> si 3
0x0000000000401019 in _start ()

code:x86:64 -----
    0x401004 <_start+4>      add     BYTE PTR [rdi+0x1], bh
    0x40100a <_start+10>     movabs  rsi, 0x402000
    0x401014 <_start+20>     mov     edx, 0x12
→   0x401019 <_start+25>     syscall
    0x40101b <_start+27>     mov     eax, 0x3c
    0x401020 <_start+32>     mov     edi, 0x0
    0x401025 <_start+37>     syscall

threads -----
[#0] Id 1, Name: "helloWorld", stopped 0x401019 in _start (), reason: SINGLE STEP
```

As we can see, we stopped at the `syscall` instruction as expected.

Tip: You can hit the `return` / `enter` empty in order to repeat the last command. Try hitting it at this stage, and you should make another 3 steps, and break at the other `syscall` instruction.

Step

The `step` or `s` command, on the other hand, will continue until the following line of code is reached or until it exits from the current function. If we run an assembly code, it will break when we exit the current function `_start`.

If there's a call to another function within this function, it'll break at the beginning of that function. Otherwise, it'll break after we exit this function after the program's end. Let us try using `s`, and see what happens:

```
shellsession
gef> step

Single stepping until exit from function _start,
which has no line number information.
Hello HTB Academy!
[Inferior 1 (process 14732) exited normally]
```

We see that the execution continued until we reached the exit from the `_start` function, so we reached the end of the program and `exited normally` without any errors. We also see that `GDB` printed the program's output `Hello HTB Academy!` as well.

Note: There's also the `next` or `n` command, which will also continue until the next line, but will skip any functions called in the same line of code, instead of breaking at them like `step`. There's also the `nexti` or `ni`, which is similar to `si`, but skips functions calls, as we will see later on in the module.

Modify

The final step of debugging is `modifying` values in registers and addresses at a certain point of execution. This helps us in seeing how this would affect the execution of the program.

Addresses

To modify values in GDB, we can use the `set` command. However, we will utilize the `patch` command in `GEF` to make this step much easier. Let's enter `help patch` in GDB to get its help menu:

```
gef> help patch
```

```
Write specified values to the specified address.
Syntax: patch (qword|dword|word|byte) LOCATION VALUES
patch string LOCATION "double-escaped string"
...SNIP...
```

As we can see, we have to provide the `type/size` of the new value, the `location` to be stored, and the `value` we want to use. So, let's try changing the string stored in the `.data` section (at address `0x402000` as we saw earlier) to the string `Patched!\n`.

We will break at the first `syscall` at `0x401019`, and then do the patch, as follows:

```
gef> break *0x401019
```

```
Breakpoint 1 at 0x401019
```

```
gef> r
```

```
gef> patch string 0x402000 "Patched!\\x0a"
```

```
gef> c
```

```
Continuing.
```

```
Patched!
```

```
Academy!
```

We see that we successfully modified the string and got `Patched!\n Academy!` instead of the old string. Notice how we used `\x0a` for adding a new line after our string.

Registers

We also note that we did not replace the entire string. This is because we only modified the characters up to the length of our string and left the remainder of the old string. Finally, the `write` system call specified a length of `0x12` of bytes to be printed.

To fix this, let's modify the value stored in `$rdx` to the length of our string, which is `0x9`. We will only patch a size of one byte. We will go into details of how `syscall` works later in the module. Let us demonstrate using `set` to modify `$rdx`, as follows:

```
gef> break *0x401019

Breakpoint 1 at 0x401019
gef> r
gef> patch string 0x402000 "Patched!\\x0a"
gef> set $rdx=0x9
gef> c
```

```
Continuing.
Patched!
```

We see that we successfully modified the final printed string and have the program output something of our choosing. The ability to modify values of registers and addresses will help us a lot through debugging and binary exploitation, as it allows us to test various values and conditions without having to change the code and recompile the binary every time.

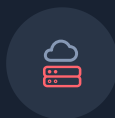
Conclusion

The ability to set breakpoints to stop the execution, step through a program and each of its instructions, examine various data and addresses at each point, and modify values when needed, enables us to do proper debugging and reverse engineering.

Whether we want to see exactly why our program is failing or understand how a program is running and what it's doing at each point, GDB becomes very handy.

For penetration testing, this process enables us to understand how a program handles input at a certain point and exactly why it's failing. This allows us to develop exploits that take advantage of such failures, as we will learn in the Binary Exploitation modules.

Connect to HTB



Pwnbox

Your own web-based Parrot Linux instance to play our labs.



Switching Pwnbox location will terminate the spawned Pwnbox.

Pwnbox Location

UK



Start Pwnbox

∞ spawns left

• Offline



Enable step-by-step solutions ⓘ

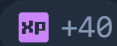
PRO

⚠ Answer questions in English to ensure an accurate response.

Question 1



+1



+40



Download the attached file, and find the hex value in 'rax' when we reach the instruction at <_start+16>?

Write your answer

 Submit

 Hint

 Download File

 Previous

Section 9 / 24

 +20

Next

Mark Complete & Next